Student Guide: Autoscaling with VMs (Stream 2)

Nov 6, 2026 · @Bayu Samudra

What you'll do

In stream 1, Lambda added a replica for every request that found no idle one, with nothing to configure. Today you run the same kind of app on **virtual machines (EC2 instances)** behind an **Application Load Balancer**, and an **Auto Scaling group** decides how many instances to run. This time *you* choose the metric, the target and the limits.

The app returns the ID of the instance that served each request (`instance_id`). Counting different IDs tells you how many instances are working, just like `env_id` did for Lambda replicas.

By the end you should be able to:

1. Build a launch template, target group, load balancer and Auto Scaling group.
2. Explain how a target tracking policy turns a metric into more or fewer instances.
3. Measure how long the Auto Scaling group takes to add and remove instances, and compare it with Lambda.
4. Explain what minimum, desired and maximum capacity do, and where HTTP 503 comes from.

Work in the **Asia Pacific (Jakarta)** Region, `ap-southeast-3`, for every step.

Same rule as stream 1: write your prediction in the results sheet *before* every run.

Cost warning: the load balancer and the instances are billed by the hour while they exist. Today's instances also run at high CPU and use detailed monitoring, which add a small extra charge. Do the cleanup at the end of the session.

Before you start

Tick each item before class.

- [ ] Stream 1 is cleaned up (no Lambda function or Provisioned Concurrency left running).
- [ ] Jakarta Region enabled and selected in the console (top right shows **Asia Pacific (Jakarta)**).

- [ ] AWS CLI v2 works: `aws sts get-caller-identity` prints your account number.
- [ ] The account has a **default VPC** in Jakarta: **VPC** console → **Your VPCs** shows one with **Default VPC = Yes**.
- [ ] k6 installed: `k6 version` works.
- [ ] A folder named `vm-scale` containing the two files from the next section.
- [ ] Windows users: run the commands in **Git Bash** or **WSL**.

You don't need Docker or a key pair today. You never log in to the instances: the load balancer is the only way in.

The two files

Create these files in your `vm-scale` folder.

user-data.sh

Every new instance runs this script once when it boots. It writes the app to `/opt/app/app.py` and starts it as a service on port 80.

Unlike the Lambda app, this one does **real CPU work**: each request runs SHA-256 on its own result a **random number of times between `min` and `max`** (default 100,000 to 200,000). Random amounts make requests uneven, like real pages where some cost more than others. On a `t3.micro` that averages roughly **0.1 s of CPU time** per request, but the exact figure depends on the processor, so you'll measure it in Step 6. The app runs **one CPU worker per vCPU**, so a `t3.micro` with 2 vCPUs serves about **20 requests per second at 100% CPU**, or 10 per second at 50%. Extra requests wait in a queue. If no worker frees up within 2 seconds, the instance answers **503 Service Unavailable**, as a real overloaded web server would.

```
#!/bin/bash
mkdir -p /opt/app
cat > /opt/app/app.py <<'EOF'
import hashlib, json, os, random, threading, time, urllib.request
from concurrent.futures import ProcessPoolExecutor
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs

CPUS = int(os.environ.get("CPUS", os.cpu_count() or 1))  # one CPU worker per
vCPU
QUEUE_WAIT = 2      # seconds a request may wait for a free CPU worker before
```

```
503
DEFAULT_MIN = 100_000     # fewest SHA-256 iterations per request
DEFAULT_MAX = 200_000     # most SHA-256 iterations per request
LIMIT = 2_000_000         # safety cap for ?max=


def burn(iterations):
    """CPU-heavy work: hash the result again and again. Returns the CPU time
used."""
    start = time.process_time()
    h = b"practicum"
    for _ in range(iterations):
        h = hashlib.sha256(h).digest()
    return round((time.process_time() - start) * 1000)


def get_instance_id():
    """Ask the EC2 metadata service (IMDSv2) which instance this is."""
    try:
        req = urllib.request.Request(
            "http://169.254.169.254/latest/api/token", method="PUT",
            headers={"X-aws-ec2-metadata-token-ttl-seconds": "300"})
        token = urllib.request.urlopen(req, timeout=2).read().decode()
        req = urllib.request.Request(
            "http://169.254.169.254/latest/meta-data/instance-id",
            headers={"X-aws-ec2-metadata-token": token})
        return urllib.request.urlopen(req, timeout=2).read().decode()
    except Exception:
        return "local"


class Handler(BaseHTTPRequestHandler):
    def send_json(self, code, data):
        body = json.dumps(data).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        global served
        url = urlparse(self.path)
```

```
        if url.path == "/health":                    # load balancer health
check
            return self.send_json(200, {"ok": True})
        qs = parse_qs(url.query)
        try:
            lo = int(qs.get("min", [DEFAULT_MIN])[0])
            hi = int(qs.get("max", [DEFAULT_MAX])[0])
        except ValueError:
            return self.send_json(400, {"error": "min and max must be whole
numbers"})
        lo, hi = sorted(min(max(n, 0), LIMIT) for n in (lo, hi))   # keep
within 0..LIMIT
        iterations = random.randint(lo, hi)          # each request gets a
random amount of work
        start = time.perf_counter()
        if not slots.acquire(timeout=QUEUE_WAIT):    # every CPU worker is
busy
            return self.send_json(503, {"instance_id": INSTANCE_ID,
                                        "error": "all CPU workers busy"})
        try:
            waited_ms = round((time.perf_counter() - start) * 1000)
            cpu_ms = pool.submit(burn, iterations).result()   # real CPU work
in a worker process
            with lock:
                served += 1
                count = served
        finally:
            slots.release()
        self.send_json(200, {"instance_id": INSTANCE_ID, "iterations":
iterations,
                             "cpu_ms": cpu_ms, "waited_ms": waited_ms,
                             "duration_ms": round((time.perf_counter() -
start) * 1000),
                             "served_by_this_instance": count})

    def log_message(self, *args):
        pass


if __name__ == "__main__":
    INSTANCE_ID = get_instance_id()
    pool = ProcessPoolExecutor(max_workers=CPUS)
    slots = threading.BoundedSemaphore(CPUS)
```

```
    lock = threading.Lock()
    served = 0
    ThreadingHTTPServer(("", int(os.environ.get("PORT", "80"))),
Handler).serve_forever()
EOF

cat > /etc/systemd/system/app.service <<'EOF'
[Unit]
Description=Practicum CPU app
After=network-online.target

[Service]
ExecStart=/usr/bin/python3 /opt/app/app.py
Restart=always

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now app
```

Response field	Meaning
`instance_id`	ID of the instance that served your request
`iterations`	How many SHA-256 rounds this request got (random, between `min` and `max`)
`cpu_ms`	CPU time those rounds took on this instance, in milliseconds
`waited_ms`	How long the request queued before a CPU worker was free
`duration_ms`	Queue time plus work time, measured on the instance
`served_by_this_instance`	How many requests this instance has served

k6.js (stream 2 version)

This replaces your stream 1 `k6.js`; it still works against Lambda too. What's new:

- **`min` / `max`**: the range of SHA-256 iterations per request on the VMs (default 100000 and 200000, change with `-e MIN=... -e MAX=...`). Each request gets a random count in that range. Lambda ignores them.
- **Pacing**: each user sends at most one request per second (`-e PACE=...`), like a real user who reads a page before clicking again. Without it, 10 users hitting a 0.1 s app would send 100 requests per second. Lambda requests already take a second, so stream 1 results don't change.
- A **`steplong`** scenario that gives the Auto Scaling group time to react, a **`wait_ms`** trend that shows how long requests queued on the instances, and a **`cpu_ms`** trend with the CPU time each request used.
- A **WHAT HAPPENED** section at the end of every run that counts each HTTP status code and explains it in one line, then prints the average CPU per request and how many requests per second one instance serves at 100% CPU and at the 50% target. It needs internet access once, to load k6's summary helper from jslib.k6.io. The `responses{status:...}` lines under **THRESHOLDS** only make those counts possible; they always pass.

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Counter, Rate, Trend } from 'k6/metrics';
import { textSummary } from 'https://jslib.k6.io/k6-summary/0.1.0/index.js';

const envHits   = new Counter('env_hits');      // tagged with the replica /
instance ID
const responses = new Counter('responses');     // tagged with the HTTP
status
const throttled = new Rate('throttled');        // share of 429 responses
(Lambda)
const status200 = new Counter('status_200');    // OK
const status429 = new Counter('status_429');    // Too Many Requests (Lambda
throttling)
const status5xx = new Counter('status_5xx');    // 502, 503, 504 (VM stream)
const statusErr = new Counter('status_other');  // anything else, 0 = network
error
const waitTime  = new Trend('wait_ms', true);   // time a request queued on
the VM
const cpuTime   = new Trend('cpu_ms', true);    // CPU time one request used
on the VM

const URL  = __ENV.URL;
```

```
const BUSY = __ENV.BUSY || '1';                  // Lambda: seconds each
request holds a replica
const MIN  = __ENV.MIN  || '100000';             // VMs: fewest SHA-256
iterations per request
const MAX  = __ENV.MAX  || '200000';             // VMs: most; each request
gets a random count in between
const PACE = Number(__ENV.PACE || 1);            // each user sends at most 1
request per PACE seconds
const VUS  = Number(__ENV.VUS || 50);

// What each status code means in this practicum (printed at the end)
const EXPLAIN = {
  200: 'OK.',
  429: 'Too Many Requests.',
  502: 'Bad Gateway.',
  503: 'Service Unavailable.',
  504: 'Gateway Timeout.',
  0:   'No response at all',
};

const scenarios = {
  warm:  { executor: 'shared-iterations', vus: 1, iterations: 10 },
  burst: { executor: 'shared-iterations', vus: VUS, iterations: VUS * 2 },
  step:  {
    executor: 'ramping-vus', startVUs: 0,
    stages: [
      { duration: '20s', target: 10 }, { duration: '40s', target: 10 },
      { duration: '20s', target: 30 }, { duration: '40s', target: 30 },
      { duration: '20s', target: 60 }, { duration: '40s', target: 60 },
      { duration: '20s', target: 10 }, { duration: '40s', target: 10 },
      { duration: '10s', target: 0 },
    ],
  },
  steplong: {
    executor: 'ramping-vus', startVUs: 0,
    stages: [
      { duration: '30s', target: 10 }, { duration: '4m', target: 10 },
      { duration: '30s', target: 30 }, { duration: '5m', target: 30 },
      { duration: '30s', target: 60 }, { duration: '5m', target: 60 },
      { duration: '30s', target: 10 }, { duration: '5m', target: 10 },
      { duration: '30s', target: 0 },
    ],
  },
```

```
  rate:  {
    executor: 'constant-arrival-rate',
    rate: 20, timeUnit: '1s', duration: '30s',
    preAllocatedVUs: 60, maxVUs: 120,
  },
};

// One always-passing threshold per status code, so the end summary can count
each one
const thresholds = {};
for (const code of Object.keys(EXPLAIN))
thresholds[`responses{status:${code}}`] = ['count>=0'];

export const options = {
  scenarios: { main: scenarios[__ENV.SCENARIO || 'burst'] },
  thresholds,
};

export default function () {
  const started = Date.now();
  const res = http.get(`${URL}/?busy=${BUSY}&min=${MIN}&max=${MAX}`);
  responses.add(1, { status: String(res.status) });
  throttled.add(res.status === 429);
  check(res, { 'status is 200': (r) => r.status === 200 });

  if (res.status === 200) {
    status200.add(1);
    const body = res.json();
    envHits.add(1, { env_id: body.instance_id || body.env_id, init_type:
body.init_type || 'vm' });
    if (body.waited_ms !== undefined) waitTime.add(body.waited_ms);
    if (body.cpu_ms !== undefined) cpuTime.add(body.cpu_ms);
  } else if (res.status === 429) {
    status429.add(1);
    sleep(1);   // back off like a well-behaved client
  } else if (res.status >= 500) {
    status5xx.add(1, { status: String(res.status) });
    sleep(1);   // back off like a well-behaved client
  } else {
    statusErr.add(1, { status: String(res.status) });
    console.warn(`HTTP ${res.status}: ${String(res.body).slice(0, 120)}`);
  }
```

```
  // pacing: like a real user, wait until PACE seconds have passed before the
next request
  const left = PACE - (Date.now() - started) / 1000;
  if (left > 0) sleep(left);
}

// Printed after k6's normal summary: what the status codes mean for this run
export function handleSummary(data) {
  const count = (name) => (data.metrics[name] ?
data.metrics[name].values.count : 0);
  const total = count('responses');
  const lines = ['', '  WHAT HAPPENED', ''];
  let explained = 0;
  for (const code of Object.keys(EXPLAIN)) {
    const n = count(`responses{status:${code}}`);
    explained += n;
    if (n === 0) continue;
    const pct = ((100 * n) / total).toFixed(1);
    lines.push(`    ${String(code).padEnd(4)} x ${String(n).padStart(5)}
(${pct}%)  ${EXPLAIN[code]}`);
  }
  if (total - explained > 0) {
    lines.push(`    other x ${String(total - explained).padStart(5)}
Unexpected status codes; see the warnings printed during the run.`);
  }
  if (total > 0 && count('responses{status:200}') === total) {
    lines.push('', '    Every request succeeded.');
  }
  const wait = data.metrics.wait_ms;
  if (wait && wait.values['p(95)'] > 0) {
    lines.push('', `    Requests queued on the instances: p95 wait
${Math.round(wait.values['p(95)'])} ms (one CPU worker per vCPU).`);
  }
  const cpu = data.metrics.cpu_ms;
  if (cpu && cpu.values.avg > 0) {
    const avg = cpu.values.avg;
    const full = Math.round(2000 / avg);   // a t3.micro has 2 vCPUs = 2000
ms of CPU per second
    lines.push('', `    Average CPU per request: ${Math.round(avg)} ms, so
one 2-vCPU instance serves about ${full} requests/s at 100% CPU`,
                   `    and about ${Math.round(full / 2)} requests/s at the
50% target.`);
  }
```

```
  lines.push('');
  return { stdout: textSummary(data, { indent: '  ', enableColors: true }) +
'\n' + lines.join('\n') + '\n' };
}
```

Scenario	What it does	Length
`burst`	`VUS` users at once, 2 requests each	Seconds
`step`	10 → 30 → 60 → 10 users, 40 s per level (same as stream 1)	About 4.5 min
`steplong`	10 → 30 → 60 → 10 users, 4–5 min per level	About 21.5 min

Part 1 · Build

You build five pieces, in this order. Traffic flows **k6 → load balancer → target group → instances**, and the Auto Scaling group adds or removes instances in the target group.

Piece	Name	What it does
Security groups	`scale-alb-sg`, `scale-web-sg`	Let the internet reach the load balancer, and only the load balancer reach the instances
Launch template	`scale-web-lt`	The recipe for a new instance: OS, size, firewall and `user-data.sh`
Target group	`scale-web-tg`	The list of instances that get traffic, plus the health check
Load balancer	`scale-alb`	One public URL that spreads requests across the target group
Auto Scaling group	`scale-web-asg`	Keeps between *min* and *max* instances running and follows the scaling policy

You can switch between console and CLI at any step. Every CLI command finds the pieces it needs by their **names** (`scale-alb-sg`, `scale-web-tg`, `scale-alb` and so on), so it doesn't matter how you made them. Keep the names exactly as written. Set the Region once before you start (this also changes your CLI's default Region):

```
aws configure set region ap-southeast-3
```

Step 1 · Security groups

Console:

1. Open **EC2** → **Security Groups** → **Create security group**.
2. Name `scale-alb-sg`, description `ALB: HTTP from anywhere`, VPC: the default VPC.
3. Inbound rules → **Add rule**: Type **HTTP**, Source **Anywhere-IPv4** → **Create security group**.
4. Create a second one: name `scale-web-sg`, description `Web: HTTP from the ALB only`, default VPC.
5. Inbound rules → **Add rule**: Type **HTTP**, Source **Custom** → choose `scale-alb-sg` → **Create security group**.

CLI:

```
# 1. Load balancer security group: HTTP from anywhere
aws ec2 create-security-group --group-name scale-alb-sg \
  --description "ALB: HTTP from anywhere" \
  --vpc-id $(aws ec2 describe-vpcs --filters Name=is-default,Values=true --
query 'Vpcs[0].VpcId' --output text)

aws ec2 authorize-security-group-ingress --protocol tcp --port 80 --cidr
0.0.0.0/0 \
  --group-id $(aws ec2 describe-security-groups --filters Name=group-
name,Values=scale-alb-sg --query 'SecurityGroups[0].GroupId' --output text)

# 2. Instance security group: HTTP from the load balancer only
aws ec2 create-security-group --group-name scale-web-sg \
  --description "Web: HTTP from the ALB only" \
  --vpc-id $(aws ec2 describe-vpcs --filters Name=is-default,Values=true --
query 'Vpcs[0].VpcId' --output text)

aws ec2 authorize-security-group-ingress --protocol tcp --port 80 \
  --group-id $(aws ec2 describe-security-groups --filters Name=group-
name,Values=scale-web-sg --query 'SecurityGroups[0].GroupId' --output text) \
  --source-group $(aws ec2 describe-security-groups --filters Name=group-
name,Values=scale-alb-sg --query 'SecurityGroups[0].GroupId' --output text)
```

Step 2 · Launch template

Console:

1. **EC2 → Launch Templates → Create launch template**.
2. Name `scale-web-lt`. Tick **Provide guidance to help me set up a template that I can use with EC2 Auto Scaling**.
3. Application and OS Images: **Amazon Linux → Amazon Linux 2023 AMI**, architecture **64-bit (x86)**.
4. Instance type: `t3.micro` (2 vCPUs).
5. Key pair: **Don't include in launch template**.
6. Network settings: Subnet **Don't include in launch template**; Security groups: `scale-web-sg`.
7. Expand **Advanced details → Detailed CloudWatch monitoring**: **Enable**. Without it, EC2 reports CPU only every 5 minutes, and the scaling policy would react much later.
8. Still in **Advanced details**, scroll to **User data** and paste the whole `user-data.sh`.
9. **Create launch template**.

CLI (in your `vm-scale` folder):

```
aws ec2 create-launch-template --launch-template-name scale-web-lt \
  --launch-template-data "{
    \"ImageId\": \"$(aws ssm get-parameter --name /aws/service/ami-amazon-
linux-latest/al2023-ami-kernel-default-x86_64 --query Parameter.Value --
output text)\",
    \"InstanceType\": \"t3.micro\",
    \"Monitoring\": {\"Enabled\": true},
    \"SecurityGroupIds\": [\"$(aws ec2 describe-security-groups --filters
Name=group-name,Values=scale-web-sg --query 'SecurityGroups[0].GroupId' --
output text)\"],
    \"UserData\": \"$(base64 < user-data.sh | tr -d '\n')\"
  }"
```

Step 3 · Target group

Console:

1. **EC2 → Target Groups → Create target group**.
2. Target type **Instances**, name `scale-web-tg`, protocol **HTTP**, port **80**, default VPC.

3. Health check path: `/health`. Expand **Advanced health check settings**: healthy threshold **2**, interval **10** seconds.
4. **Next** → don't register any targets (the Auto Scaling group does that) → **Create target group**.
5. On the `scale-web-tg` page, open the **Attributes** tab (between **Health checks** and **Tags**) → **Edit** → under **Target deregistration management**, set **Deregistration delay** to **30** seconds → **Save changes**. The default of 300 s would make scale-in look much slower than it is.

CLI:

```
aws elbv2 create-target-group --name scale-web-tg --protocol HTTP --port 80 \
  --vpc-id $(aws ec2 describe-vpcs --filters Name=is-default,Values=true --
query 'Vpcs[0].VpcId' --output text) \
  --health-check-path /health --health-check-interval-seconds 10 --healthy-
threshold-count 2

aws elbv2 modify-target-group-attributes \
  --target-group-arn $(aws elbv2 describe-target-groups --names scale-web-tg
--query 'TargetGroups[0].TargetGroupArn' --output text) \
  --attributes Key=deregistration_delay.timeout_seconds,Value=30
```

Step 4 · Load balancer

Console:

1. **EC2** → **Load Balancers** → **Create load balancer** → **Application Load Balancer** → **Create**.
2. Name `scale-alb`, scheme **Internet-facing**, IP address type **IPv4**.
3. Network mapping: default VPC; tick **every** Availability Zone (at least two are required).
4. Security groups: remove `default`, choose `scale-alb-sg`.
5. Listeners: **HTTP : 80** → Default action **Forward to** `scale-web-tg`.
6. **Create load balancer**. Copy its **DNS name** and save it in your terminal:

```
URL=http://scale-alb-xxxxxxxx.ap-southeast-3.elb.amazonaws.com
```

CLI:

```
aws elbv2 create-load-balancer --name scale-alb \
  --subnets $(aws ec2 describe-subnets --filters Name=default-for-
az,Values=true --query 'Subnets[].SubnetId' --output text) \
  --security-groups $(aws ec2 describe-security-groups --filters Name=group-
name,Values=scale-alb-sg --query 'SecurityGroups[0].GroupId' --output text)

aws elbv2 create-listener --protocol HTTP --port 80 \
  --load-balancer-arn $(aws elbv2 describe-load-balancers --names scale-alb -
-query 'LoadBalancers[0].LoadBalancerArn' --output text) \
  --default-actions Type=forward,TargetGroupArn=$(aws elbv2 describe-target-
groups --names scale-web-tg --query 'TargetGroups[0].TargetGroupArn' --output
text)

# print the DNS name, then save it as URL (see the console steps above)
aws elbv2 describe-load-balancers --names scale-alb --query
'LoadBalancers[0].DNSName' --output text
```

The URL is **http**, not https.

Step 5 · Auto Scaling group and scaling policy

The policy is the heart of today. **Target tracking** keeps one metric near a value you choose. You'll track **average CPU utilization** across the group with a target of **50%.** One instance at 50% CPU serves about 10 requests per second, so 10 users need about 1 instance, 30 users about 3, and 60 users about 6. The other 50% is headroom: it absorbs a burst while new instances boot.

Console:

1. **EC2 → Auto Scaling Groups → Create Auto Scaling group**.
2. Name `scale-web-asg`, launch template `scale-web-lt` (version **Latest**) → **Next**.
3. Network: default VPC, choose **every** Availability Zone subnet → **Next**.
4. Load balancing: **Attach to an existing load balancer → Choose from Application or Network Load Balancer target groups →** `scale-web-tg`. The list shows **target groups**, not load balancers: picking `scale-web-tg` connects the group to `scale-alb`. It only lists target groups in the **same VPC** you chose in item 3, so if the list is empty, click **Previous** and choose the default VPC.
5. Health checks: turn on **Elastic Load Balancing health checks**, grace period **60** seconds.

6. Additional settings: tick **Enable group metrics collection within CloudWatch**, and set **Default instance warmup** to **60** seconds → **Next**.
7. Group size: desired **1**, minimum **1**, maximum **6**.
8. Automatic scaling: **Target tracking scaling policy**. Name `cpu-target`, metric type **Average CPU utilization**, target value **50** → **Next**.
9. Skip notifications and tags → **Create Auto Scaling group**.

CLI:

```
# 1. The group: 1 to 6 instances, registered in scale-web-tg
aws autoscaling create-auto-scaling-group --auto-scaling-group-name scale-
web-asg \
  --launch-template "LaunchTemplateName=scale-web-lt,Version=\$Latest" \
  --min-size 1 --max-size 6 --desired-capacity 1 \
  --vpc-zone-identifier "$(aws ec2 describe-subnets --filters Name=default-
for-az,Values=true --query 'Subnets[].SubnetId' --output text | tr '\t' ',')"
\
  --target-group-arns $(aws elbv2 describe-target-groups --names scale-web-tg
--query 'TargetGroups[0].TargetGroupArn' --output text) \
  --health-check-type ELB --health-check-grace-period 60 \
  --default-instance-warmup 60

# 2. Send group metrics to CloudWatch every minute
aws autoscaling enable-metrics-collection --auto-scaling-group-name scale-
web-asg --granularity 1Minute

# 3. Target tracking: keep average CPU near 50%
aws autoscaling put-scaling-policy --auto-scaling-group-name scale-web-asg \
  --policy-name cpu-target --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{"TargetValue": 50,
"PredefinedMetricSpecification": {"PredefinedMetricType":
"ASGAverageCPUUtilization"}}'
```

Step 6 · First call

1. Wait 2–3 minutes for the first instance to boot. **EC2** → **Target Groups** → `scale-web-tg` → **Targets** should show one target as **healthy**.
2. Open `<your URL>/` in a browser, or run `curl "$URL/"`. Refresh a few times: `iterations` and `cpu_ms` change on every request.

3. Measure what one request really costs: `k6 run -e URL=$URL -e SCENARIO=warm -e MIN=100000 -e MAX=200000 k6.js`. The last lines of WHAT HAPPENED give the average CPU per request and how many requests per second one instance serves at 100% and at 50% CPU. The experiments assume about 100 ms. Every experiment below uses the same range, so the results compare. If your average is far off (say 150 ms), shrink MIN and MAX by the same factor (100 ÷ 150: -e MIN=67000 -e MAX=133000) and use those two values in every experiment instead, so the predictions still hold.

✅ Check: you get JSON with an `instance_id` that starts with `i-`. Refresh a few times: it's always the same instance, because there is only one.

Look at what the policy created: open **CloudWatch** → **Alarms**. You'll find two alarms whose names start with `TargetTracking-scale-web-asg`: one ending in **AlarmHigh** (adds instances) and one ending in **AlarmLow** (removes them). Open each one and write down how many minutes of data it needs before it fires. You'll use these numbers in Experiment 2.

Watching instances

You'll use two views. The **Activity** tab gives exact times, and a **CloudWatch graph** shows the shape over time.

Activity tab: exact times

EC2 → **Auto Scaling Groups** → `scale-web-asg` → **Activity**. Every launch or termination is listed with its **start time**, **end time** and **cause**, for example "an alarm … AlarmHigh … changing the desired capacity from 1 to 3".

To measure how fast the group scales, note the time k6 raised the load and the time the new instance's activity ended. Then add the time it took to become **healthy** in the target group.

CloudWatch graph: the shape

1. **CloudWatch** → **Metrics** → **All metrics**.
2. **Auto Scaling** → **Group Metrics** → tick `scale-web-asg` **GroupInServiceInstances** and **GroupDesiredCapacity**.
3. **EC2** → **By Auto Scaling Group** → tick `scale-web-asg` **CPUUtilization**.

4. Open **Graphed metrics**: set every **Period** to **1 minute**. Set the statistic to **Maximum** for the two Group metrics and **Average** for CPUUtilization. Move CPUUtilization to the right Y axis.
5. Time range: **30m** or **1h**. Refresh after each stage. Metrics arrive 1–2 minutes late.

Line	What it tells you
GroupDesiredCapacity	How many instances the policy *asked for*
GroupInServiceInstances	How many are actually running
CPUUtilization	The metric the policy watches; compare it with your target of 50%

The gap between **desired** and **in service** is the boot time. Lambda had no such gap you could see.

From k6: `status_5xx` counts users who were turned away, and `wait_ms` shows how long requests queued for a free CPU worker.

Experiment 1 · One instance vs a burst

Question: what happens when a burst arrives faster than the Auto Scaling group can react?

In stream 1, 50 users at once made Lambda start about 50 replicas within seconds. Here, one instance has 2 vCPUs and serves about 20 requests per second at full CPU, and the Auto Scaling group looks at CPU only once a minute.

Predict: 30 users at once, each sending one request per second. Can one instance keep up? How high will CPU go, and how long will requests queue? Will anyone get a 503?

Run:

```
k6 run -e URL=$URL -e SCENARIO=burst -e VUS=30 \
  -e MIN=100000 -e MAX=200000 k6.js
```

Observe and record:

- How many different `instance_id` values came back. Run the command again with `--out json=out.json` and count them as in stream 1, or check the **Activity** tab for any launch.

- `status_200` and `status_5xx` in the k6 summary.
- `wait_ms` (p95) and `http_req_duration` (p95).

Think about it:

1. One instance serves about 20 requests per second, and 30 users asked for 30. Where did the extra requests go?
2. The whole burst was over in a few seconds. Why didn't the Auto Scaling group add an instance?
3. Compare with stream 1, Experiment 1b. Which platform protected these 30 users better, and why?

Experiment 2 · Step load

Question: do instances follow demand up and down, and how far behind are they?

2a · The same step as stream 1 (about 4.5 min)

This is exactly the load that made Lambda form a clean staircase of replicas.

Predict: at 60 users, how many instances will be in service before the test ends?

```
k6 run -e URL=$URL -e SCENARIO=step \
  -e MIN=100000 -e MAX=200000 k6.js
```

Observe: the **Activity** tab and the CloudWatch graph during the run and for 5 minutes after. Did any instance arrive *before* the load had already dropped?

2b · A step long enough for the group to react (about 21.5 min)

Each level now lasts 4–5 minutes. Before you start, write down how many instances are in service. 2a may have left extra ones running.

Predict: the table works out how many instances the policy should ask for. Check the arithmetic, then add your own guess in the results sheet: how many **minutes** each step up will take to arrive.

Stage	Users	CPU work per second (users × about 0.1 s)	Instances at 50% CPU (÷ 1 per instance)	Lambda replicas in stream 1
1	10	1 CPU-second	1 (maybe 2)	About 10

Stage	Users	CPU work per second (users × about 0.1 s)	Instances at 50% CPU (÷ 1 per instance)	Lambda replicas in stream 1
2	30	3 CPU-seconds	3	About 30
3	60	6 CPU-seconds	6 (the maximum)	About 60
4	10	1 CPU-second	1, eventually	About 10

One instance has 2 vCPUs, so at 50% it does 1 CPU-second of work per second.

```
K6_WEB_DASHBOARD=true k6 run -e URL=$URL -e SCENARIO=steplong \
  -e MIN=100000 -e MAX=200000 k6.js
```

Open `http://localhost:5665` to see the load live, next to the CloudWatch graph.

Observe and record:

- For each step **up**: the time the users increased, and the time the new instances were in service and **healthy**. That difference is your **scale-out time**.
- For the step **down** to 10 users: how many minutes until the first instance was terminated. That is your **scale-in time**. Check whether it fits the AlarmLow numbers you wrote down in Part 1.
- `status_5xx` and `wait_ms` during the minutes when instances were still booting.

Think about it:

1. In 2a, Lambda would have followed every step within seconds. Why does the Auto Scaling group need minutes?
2. Which takes longer, scaling out or scaling in? Why do you think AWS chose that?
3. Who got the 503s: users at the start of each step, or users in the middle? What does that tell you about the cost of slow scaling?
4. Lambda needed no policy. What did you have to decide here, and what happens if you choose the target value badly?

Experiment 3 · Pre-warmed instances

Question: can we avoid the wait by starting instances *before* the traffic comes?

This is the VM version of Provisioned Concurrency. You raise the **minimum** and **desired** capacity, so the instances are booted and healthy before the burst.

Predict: with 6 instances ready (12 vCPUs, about 120 requests per second at full CPU), what happens to 60 users at once? And to 200?

Step 1 · Raise the minimum (wait 3–4 min)

- **Console: Auto Scaling Groups** → `scale-web-asg` → **Details** → **Group details** → **Edit** → Desired **6**, Min **6**, Max **6** → **Update**.
- **CLI:** `aws autoscaling update-auto-scaling-group --auto-scaling-group-name scale-web-asg --min-size 6 --desired-capacity 6`

Wait until **Target Groups** → `scale-web-tg` → **Targets** shows **6 healthy** targets.

Step 2 · Run

```
# 3a: about half of what 6 instances can do
k6 run -e URL=$URL -e SCENARIO=burst -e VUS=60 \
  -e MIN=100000 -e MAX=200000 k6.js

# 3b: more than 6 instances can serve
k6 run -e URL=$URL -e SCENARIO=burst -e VUS=200 \
  -e MIN=100000 -e MAX=200000 k6.js
```

Observe and record: `status_5xx`, `wait_ms` (p95) and `http_req_duration` (p95) for both runs. Compare 3a with Experiment 1, where 30 users hit a single instance.

Step 3 · Set it back

- **Console:** Edit the group again → Desired **1**, Min **1**, Max **6** → **Update**.
- **CLI:** `aws autoscaling update-auto-scaling-group --auto-scaling-group-name scale-web-asg --min-size 1 --desired-capacity 1`

Lowering *desired* terminates instances right away. Compare that with how slowly the policy scaled in during Experiment 2.

Think about it:

1. Why was 3a fast for everyone? Why did 3b still produce slow answers or 503s?
2. Stream 1 had Provisioned Concurrency. What is the same here, and what is different?
3. Six instances all day cost six times as much as one. When is that worth it? What could you do instead for a peak you know is coming, such as 9 AM every day?

Experiment 4 · Hitting the ceiling

Question: what do users get when the group is not allowed to grow any more?

The **maximum capacity** is the VM version of reserved concurrency. You'll set it to 2: 4 vCPUs, about 40 requests per second at full CPU.

Predict: 120 users at once against 2 instances. How many get a 200? What do the others get, and is it the same error as Lambda's?

Step 1 · Set the ceiling (wait 2–3 min)

To save time, start at the ceiling with 2 instances already running.

- **Console:** Edit the group → Desired **2**, Min **2**, Max **2** → **Update**. Wait for **2 healthy** targets.
- **CLI:** `aws autoscaling update-auto-scaling-group --auto-scaling-group-name scale-web-asg --min-size 2 --max-size 2 --desired-capacity 2`

Step 2 · Run

```
k6 run -e URL=$URL -e SCENARIO=burst -e VUS=120 \
  -e MIN=100000 -e MAX=200000 k6.js
```

Observe and record: `status_200`, `status_5xx`, `wait_ms` (p95). In the **Activity** tab, check whether the policy even tried to add instances.

Bonus (2 min): set Desired, Min and Max all to **0** and wait until no targets are registered. Then run `curl -i "$URL/?busy=1"`. This 503 comes from the **load balancer itself**, because it has no instance to send the request to.

Step 3 · Set it back

- **Console:** Edit the group → Desired **1**, Min **1**, Max **6** → **Update**.
- **CLI:** `aws autoscaling update-auto-scaling-group --auto-scaling-group-name scale-web-asg --min-size 1 --max-size 6 --desired-capacity 1`

Think about it:

1. Lambda answered 429 at its ceiling. What did you get here, and who sent it: the instance or the load balancer?
2. Lambda rejected extra requests in milliseconds. How long did a rejected user wait here before getting the 503? Which is better for the user?

3. Why would anyone set a low maximum on purpose?

Experiment 5 · A target set too high (take-home)

Question: what changes if the CPU target is 90% instead of 50%?

A higher target means fewer instances and a smaller bill. But each instance runs close to full, so there is little room to absorb a burst while new instances boot.

Predict: with a 90% target, how many instances will run at 30 and at 60 users? How long will requests queue, compared with Experiment 2b?

Step 1 · Raise the target to 90%

- **Console:** `scale-web-asg` → **Automatic scaling** → select `cpu-target` → **Actions** → **Edit** → target value **90** → **Update**.
- **CLI:** running `put-scaling-policy` with the same policy name replaces it.

```
aws autoscaling put-scaling-policy --auto-scaling-group-name scale-web-asg \
  --policy-name cpu-target --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{"TargetValue": 90,
"PredefinedMetricSpecification": {"PredefinedMetricType":
"ASGAverageCPUUtilization"}}'
```

Step 2 · Run

Start from 1 instance and run the long step:

```
k6 run -e URL=$URL -e SCENARIO=steplong \
  -e MIN=100000 -e MAX=200000 k6.js
```

Observe and record: instances in service at each level, the highest **CPUUtilization**, `wait_ms` (p95) and `status_5xx`. Put them next to your Experiment 2b numbers.

Step 3 · Put the target back to 50%

Use **Edit** again with target value **50**, or rerun the command above with `"TargetValue": 50`.

Think about it:

1. How many instances did the 90% target save, and what did users pay for it?

2. When an instance is nearly full, why do *all* requests get slower, not only the extra ones?
3. Lambda had no target to choose. Is that an advantage or a limitation? Give one example of each.

Cleanup

Do this before you leave. The load balancer and every instance cost money each hour, even with no traffic. Delete in this order, because each piece is used by the next one in the list.

Console:

1. **EC2** → **Auto Scaling Groups** → select `scale-web-asg` → **Actions** → **Delete** → type `delete`. This terminates its instances and removes the scaling policy and its alarms. Wait until **Instances** shows them as terminated.
2. **Load Balancers** → select `scale-alb` → **Actions** → **Delete load balancer** → type `confirm`.
3. **Target Groups** → select `scale-web-tg` → **Actions** → **Delete**.
4. **Launch Templates** → select `scale-web-lt` → **Actions** → **Delete template**.
5. **Security Groups** → delete `scale-web-sg` first, then `scale-alb-sg`. If it says the group is still in use, wait a minute for the instances and load balancer to finish deleting.

CLI (finds everything by name, so it works however you built it):

```
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name scale-
web-asg --force-delete

aws elbv2 delete-load-balancer \
  --load-balancer-arn $(aws elbv2 describe-load-balancers --names scale-alb -
-query 'LoadBalancers[0].LoadBalancerArn' --output text)

# wait about a minute, then:
aws elbv2 delete-target-group \
  --target-group-arn $(aws elbv2 describe-target-groups --names scale-web-tg
--query 'TargetGroups[0].TargetGroupArn' --output text)

aws ec2 delete-launch-template --launch-template-name scale-web-lt

# wait until the instances are terminated, then:
aws ec2 delete-security-group \
  --group-id $(aws ec2 describe-security-groups --filters Name=group-
name,Values=scale-web-sg --query 'SecurityGroups[0].GroupId' --output text)
aws ec2 delete-security-group \
  --group-id $(aws ec2 describe-security-groups --filters Name=group-
name,Values=scale-alb-sg --query 'SecurityGroups[0].GroupId' --output text)
```

- [ ] Auto Scaling group deleted and all its instances terminated
- [ ] Load balancer deleted
- [ ] Target group deleted
- [ ] Launch template deleted
- [ ] Both security groups deleted
- [ ] **CloudWatch** → **Alarms** shows no `TargetTracking-scale-web-asg` alarms left

Results sheet

Fill the predicted columns before each run.

Exp	Run	Setting	Predicted instances	Measured instances	Time to scale (min)	5xx count	p95 latency
1	burst	30 users, 1 instance					
2a	step	Same as stream 1					

Exp	Run	Setting	Predicted instances	Measured instances	Time to scale (min)	5xx count	p95 latency
2b	steplong: 10 users						
2b	steplong: 30 users						
2b	steplong: 60 users						
2b	steplong: back to 10						
3a	burst	60 users, min 6					
3b	burst	200 users, min 6					
4	burst	120 users, max 2					
5	steplong	CPU target 90%					

Autoscaling framework

The Lambda column holds the stream 1 answers. Fill in the VM column from today's experiments.

Question	Lambda (stream 1)	VM + Auto Scaling (today)
What is the scaling unit?	A replica, one request at a time	
What triggers scaling?	A request with no idle replica	
How fast does new capacity appear?	Seconds; instant if pre-warmed	
What is the limit?	Account limit or reserved concurrency	
How does it scale in?	Idle replicas are reclaimed automatically	

Question	Lambda (stream 1)	VM + Auto Scaling (today)
What do users get past the limit?	429, in milliseconds	
What did you have to configure?	Nothing to scale out	

Reflection (for your lab report)

1. Using your measured times from Experiment 2b, explain why the Auto Scaling group is slower than Lambda in both directions.
2. Name one thing the VM setup lets you control that Lambda does not, and one situation where that control matters.
3. A ticket-booking site opens sales at 10:00 and gets 50 times its normal traffic for 20 minutes. Which stream's approach would you use, and with which settings? Use your results to justify it.

Troubleshooting

Problem	Likely cause	Fix
URL returns **503** right after setup	No instance registered or healthy yet	Wait 2–3 min; check **Target Groups → Targets**
Targets stay **unhealthy**	Health check path wrong, or `scale-web-sg` doesn't allow the load balancer	Path must be `/health`; inbound HTTP source must be `scale-alb-sg`
URL returns **502 Bad Gateway**	The app isn't running on the instance (user data failed)	Check the launch template's user data is the whole `user-data.sh`, starting with `#!/bin/bash`; then terminate the instance so the group replaces it
Browser or k6 can't connect at all	Using `https://`, or `scale-alb-sg` missing the HTTP rule	Use `http://`; check the inbound rule allows port 80 from anywhere

Problem	Likely cause	Fix
Activity tab shows a failed launch	Instance type not offered in one Availability Zone, or an account limit	Read the failure message; remove that zone's subnet from the group, or ask your instructor
`put-scaling-policy` fails with a ResourceLabel error	The load balancer or target group has a different name, so the lookup found nothing	Names must be exactly `scale-alb` and `scale-web-tg`; check them in the console
No instances added during Experiment 2b	Policy missing, max capacity is 1, or detailed monitoring is off in the launch template (CPU then arrives only every 5 minutes)	Check **Automatic scaling** and **Group details** (max 6)
Security group won't delete	Still in use by an instance or the load balancer	Wait a minute after deleting them, then try again
k6 says `URL` is undefined	Variable not set in this terminal	`echo $URL`; set it again as in Step 4

Still stuck? Paste the exact error message into the class sheet.